

CYBR/CMSC 688

Software Reverse Engineering

Monday 7:10 pm - 9:40 pm

Dr. Daniel Guernsey

danpg102@umbc.edu

Online Lectures on Google Meet

“Office hours” after class and by appointment

This course will introduce you to the foundations, tools, techniques, and applications of Software Reverse Engineering (SRE). SRE has applications in both software development and information security. Many popular programs tend to be closed source, usually to protect intellectual property. This makes review by parties other than the original developers difficult, and prevents potential competitors from building interoperable programs. Even some popular open-source packages are not thoroughly reviewed for security flaws. Thus, SRE is a fundamental and valuable skill in cyber security.

Objectives

We explore a variety of analysis techniques used to understand the inner workings of a binary program or system of programs. This includes systems analysis, static code analysis, and dynamic code analysis. Students will gain hands-on experience using reverse engineering tools including system monitors, disassemblers, and debuggers, to perform important tasks such as evaluating program security, constructing interoperable software, and understanding the nature of malware. By the end of the course, you will learn how to understand and manipulate computer software without referring to its original source code.

Tentative Lecture Schedule

The lectures follow the textbook closely. Generally, each lesson is given one or two weeks, favoring the early fundamental chapters. Unless noted otherwise, each lesson is accompanied by a lab. For lessons with labs, you are expected to read the lecture notes and book chapter before class.

Topic	Lab
9 Sep C & x86 Refresher	Poker: Shuffling cards and scoring hands using Intel x86-64 assembly and C
16 Sep Ch 1: Foundations	SRE environment software installation and set-up
23 Sep Ch 2: Low-Level Software	Compiler Experiments: Locating implementations of C code in machine-generated assembly
30 Sep Ch 4: Reversing Tools	<i>No lab.</i> Day to work on other labs
7 Oct Ch 5: Beyond the Documentation	Tic-Tac-Toe: Reversing functions and structures to understand an undocumented Application Programming Interface, and developing a substitute library in a software system
14 Oct Ch 6: Deciphering File Formats	Maze Escape: Reversing message formats and sequences to understand an undocumented network protocol, and developing a substitute client in a networked multi-player game
21 Oct Lab Day	Day to work on labs and prepare for mid-term
28 Oct Mid-term Exam	
4 Nov Ch 7: Auditing Program Binaries	Examining a vulnerable network server, developing an exploit for the vulnerability, and devising mitigations
11 Nov Ch 8: Reversing Malware	Using a protected SRE environment to catalog a malware sample's command set
18 Nov Ch 9: Piracy and Copy Protection	<i>No lab.</i> Day to work on other labs
25 Nov Ch 10: Antireversing Techniques	Devising and implementing anti-RE techniques to protect a software-based license key checker
2 Dec Ch 11: Breaking Protections	Evaluating software-based copy protections, and defeating anti-RE techniques
9 Dec Student Topics	Possible student presentations, lab day, and/or prepare for final
16 Dec Final Exam	

Materials

The text *Reversing: Secrets of Reverse Engineering* by Eldad Eilam (published by Wiley) is required for this course. You will also need a moderately powerful computer that you can bring to class with you. Lab materials are available for Windows and Linux. For those who prefer, some lab materials are available for macOS x86_64 — some also for arm64, but with big caveats. The instructor will use the latest consumer desktop edition of Windows and/or the latest distribution of Ubuntu Desktop Linux on the “normal” upgrade path. To have access to both in the classroom, a computer capable of running a virtual machine is highly recommended.

Optional Reading Material

- Bruce Dang, et al., *Practical Reverse Engineering: x86, x64, ARM, Windows Kernel, Reversing Tools, and Obfuscation*, Wiley, February 17, 2014.
- Chris Eagle, *The IDA Pro Book: The Unofficial Guide to the World’s Most Popular Disassembler*, No Starch Press, July 14, 2011.
- Jeff Duntemann, *Assembly Language Step-by-step: Programming with DOS and Linux*, Wiley, April 21, 2008.
- Andrew Huang, *Hacking the Xbox: An Introduction to Reverse Engineering*, No Starch Press, July 1, 2003.
- Kris Kaspersky, *Hacker Disassembling Uncovered: Powerful Techniques to Safeguard your Programming*, A-List Publishing, April 1, 2003.

Course Structure and Grading

Labs 1 and 2 will be completed individually. Labs 3 and on will be completed in groups of two to four students (depending on class size). Labs may be worked on during class time or at home, and they must be completed with a write-up (unless otherwise noted) and submitted by the posted due date. Late submissions, if accepted, are subject to point deductions. If you anticipate circumstances which will prevent you from completing an assignment on time, please notify me before the due date.

Exams are completed individually during class time and usually consist of two portions: a written portion and a practical portion. Unlike the written portion, the practical portion permits access to the textbook, notes, computer, Internet, etc.

The course is graded with a heavy emphasis on labs. There are 9 labs, each comprising 3 points for technical writing and 7 points for lab-specific technical accomplishments.

Labs (9)	60%
Mid-term	15%
Final	25%

Course-specific Policies

Abuse of the knowledge or experience you gain in the course may subject you to discipline under UMBC policy and/or criminal prosecution. Do not expect your status as a student to protect you if you break the law! Hacking into campus computers (other than systems approved for such a purpose) is a violation of UMBC policy, and may result in disciplinary action possibly including expulsion.

Academic dishonesty of any kind will be handled in accordance with University policy.

By enrolling in this course, each student assumes the responsibilities of an active participant in UMBC's scholarly community, in which everyone's academic work and behavior are held to the highest standards of honesty. Cheating, fabrication, plagiarism, and helping others to commit these acts are all forms of academic dishonesty, and they are wrong. Academic misconduct could result in disciplinary action that may include, but is not limited to, suspension or dismissal. To read the full Student Academic Conduct Policy, consult the UMBC Student Handbook, the Faculty Handbook, or the UMBC Policies section of the UMBC Directory.
<https://catalog.umbc.edu/content.php?catoid=14&navoid=718>.

If the integrity of your work in this course is challenged, *you* are responsible for demonstrating proof that the work submitted is your own. In particular, we have a ZERO TOLERANCE policy for the abuse of AI technologies, including but not limited to, ChatGPT, CoPilot, Gemini, Perplexity, etc. A good starting point is to enable version tracking in GDocs, Word, or Pages so that your writing activities/progress during the semester can be logged if necessary. Keeping copies of research notes, scribbles, and related material may be helpful, too.

I will evaluate your submission. However, to ensure academic and professional integrity, I reserve the right to hold a one-on-one oral viva (defense) of labs or exams deemed questionable at the end of the semester to determine your knowledge and mastery of the topic versus the material submitted. Failing that viva will result in an 'F' for your submission (and subsequently this course) and an Academic Integrity violation report filed with the Graduate School.

BY REMAINING IN THIS COURSE YOU UNDERSTAND AND AGREE TO THESE TERMS.

University Policies

As required by federal law, here are UMBC's various ADA and Title IX notices: <https://oei.umbc.edu/sample-title-ix-responsible-employee-syllabus-language/>.

Absences, Group Work, and Illness

If you know you are going to be absent from class (for non-illness reasons) please let me and your lab group(s) know as soon as you know. I realize many of us are working professionals and may not have complete control of our own schedule; nevertheless, notifying me ahead of time will help us accommodate you.

This semester's course is conducted online only, except for exams, which will be on campus. If you would like to make an appointment for office hours, please email me, and we'll arrange a time. If you feel sick, have tested positive for an airborne contagion, or have come in contact

with anyone who has, please *do not* arrange in-person meetings with me or your classmates. If you fall ill, please take care of yourself and let me and your group know. Once you are well enough, message me, and we will work out arrangements to catch you up, if necessary.

Lab groups may meet in person at their option. This is an online class, so any student requesting to meet online only *must* be accommodated by the rest of the group. Furthermore, the group cannot exclude or otherwise penalize a member who attends meetings virtually. If the group cannot agree on a policy for meeting in person, then please default to online meetings.

Apple Silicon / Windows on ARM

If you are also the proud owner of an Apple Silicon or Snapdragon device, your computer is using the ARM64 architecture, which may cause some difficulty with this x86-centric class. If you do not have access to an x86 device, here are a few options:

1. **Purchase a cheap x86 laptop.** I understand this is not ideal, but there are several relatively inexpensive options available. *Do not* buy a Chromebook! Find a device that ships with Windows or Linux, and check that it has an Intel or AMD processor, not Qualcomm.
2. **Use x86 virtualization software.** Some options to try are VMware, Parallels Desktop, UTM, etc. Cost and mileage may vary. Be certain the software can run on ARM and properly host an x86 guest. Running Windows on ARM as a guest on Apple Silicon, e.g., might not improve the situation.
3. **Utilize university computing resources.** The university has computer labs with x86 machines available. Some of these resources may even be accessible remotely. Check with the owners or maintainers of those resources about use for this class.
4. **Borrow a friend's computer.** This is not ideal, because it may not always be available to you, and you'll have less control of your work. Always keep back-ups of your work on your own computer or removable drive in case your friend has to re-install Windows or something.
5. **Complete the course using x86 on macOS via Rosetta 2 or on Windows via Prism.** Some of the course materials are available for Intel-based macOS, and they may run on Apple Silicon using Rosetta 2. The course materials for x86 Windows may “just work” on Windows on ARM. If you intend to pick this option, please let me know sooner than later, so I can prepare the materials.
6. **Complete the course using ARM64 natively.** I've been considering porting the course materials to native ARM. If you intend to pick this option, please let me know sooner than later. ARM is a different assembly language from x86. I have not taught ARM before, so you will be well off the beaten path, and class lab time will still be taught in x86. On the other hand, ARM is the dominant platform in the mobile market, and the desktop market is moving toward it, so learning it is certainly valuable.
7. **Drop the course.** I don't mean to be cheeky. If you are not willing to try one of the above options or work out a solution on your own, you *will not* be successful in the course.